

Esame Programmazione di Sistema / API Programming

Data: 2024-07-15

QUESTIONS

Domanda 1

Si considerino le seguenti strutture dati e rispettive porzioni di codice. Per ciascuna di esse si indichi la dimensione di memoria allocata nello stack e nello heap.

```
1 use std::rc::Rc;
2 let rc: Rc<u64> = Rc::new(42);
3 let rc2 = rc.clone();
4 let wk = Rc::downgrade(&rc);
```

Listing 1: Struttura 1

```
1 let mut vector = Vec::<u64>::with_capacity(8);
2 for i in 0..5 {
3     vector.push(i);
4 }
5 let vslice = &vector[1..3];
```

Listing 2: Struttura 2 e codice di supporto

Domanda 2

Si consideri il programma seguente che riporta la numerazione delle linee di codice. Spiegare i problemi di questo codice e come risolverli.

```
1 use std::sync::{Arc, Mutex, Condvar};
2 use std::thread;
3
4 fn main() {
5     let data = Arc::new((Mutex::new(Vec::new()), Condvar::new()));
6
7     let data_clone1 = Arc::clone(&data);
8     let data_clone2 = Arc::clone(&data);
9
10    let writer = thread::spawn(move || {
11        let (lock, cvar) = &*data_clone1;
12        for i in 1..=5 {
13            thread::sleep(std::time::Duration::from_secs(1));
14
15            let mut data = lock.lock().unwrap();
16            data.push(i);
17            cvar.notify_all();
18        }
19    });
20
21    let reader1 = thread::spawn(move || {
22        let (lock, cvar) = &*data_clone2;
23        loop {
```

```

24         let mut data = lock.lock().unwrap();
25
26         data = cvar.wait(data).unwrap();
27
28         if let Some(value) = data.pop() {
29             println!("Reader 1: read {}", value);
30         }
31         else {
32             println!("{}", (1));
33         }
34     }
35 });
36
37 let reader2 = thread::spawn(move || {
38     let (lock, cvar) = &*data;
39     loop {
40         let mut data = lock.lock().unwrap();
41
42         data = cvar.wait(data).unwrap();
43
44         if let Some(value) = data.pop() {
45             println!("Reader 2: read {}", value);
46         }
47         else {
48             println!("{}", (2));
49         }
50     }
51 });
52
53 writer.join().unwrap();
54 reader1.join().unwrap();
55 reader2.join().unwrap();
56 }

```

Domanda 3

Si implementi in RUST la struct `CountDownLatch` che permette a uno o più thread di attendere che un gruppo di operazioni eseguite da altri thread siano eseguite. Essa incapsula un contatore ed offre i seguenti tre metodi thread-safe (oltre alla propria funzione costruttrice) che devono essere implementati:

```

1 fn new(n: usize) -> Self
2 // inizializza la struttura, impostando ad n il contatore
3
4 fn count_down(&self)
5 // decrementa il contatore, se maggiore di 0
6
7 fn wait(&self)
8 // blocca l'esecuzione del chiamante senza consumare cicli di cpu, finche' il
   contatore non diventa zero
9
10 fn wait_timeout(&self, d: Duration) -> std::sync::WaitTimeoutResult
11 // blocca l'esecuzione del chiamante senza consumare cicli di cpu, in attesa che
   il contatore raggiunga 0 per una durata massima pari a d, restituendo il
   risultato dell'attesa.

```

Domanda 4

Si descriva nel dettaglio il comportamento di questo programma. Si spieghi l'errore di compilazione che viene generato. Che cosa deve essere modificato per permetterne la compilazione in modo da ottenere la visualizzazione di "Numero: 4"?

```
1 fn main() {
2     let mut data = vec![1, 2, 3, 4, 5];
3     data.push(60);
4     let mut process_data = move || {
5         data.push(50);
6         let count = data.iter().filter(|&x| x % 2 == 0).count();
7         println!("Numero: {:?}", count);
8     };
9
10    data.push(40);
11
12    process_data();
13
14    data.push(30);
15
16 }
```

SOLUTIONS

Soluzione Domanda 1

Struttura 1

- **Stack:** `rc` (8 byte), `rc2` (8 byte) e `wk` (8 byte), per un totale di 24 byte. Ognuno di essi è un puntatore (smart pointer) al dato allocato nello heap.
- **Heap:** un solo blocco di dati allocato al momento della `Rc::new(42)`. Questo blocco contiene lo `strong_counter` (8 byte), il `weak_counter` (8 byte) e il dato di tipo `u64` (8 byte) contenente il numero 42, per un totale di 24 byte. In questa fase avremo `strong_counter = 2` e `weak_counter = 1`.

Struttura 2

- **Stack:**
 - `vector`: *fat pointer* allo heap composto da 3 campi (8 byte per il `size`, 8 byte per la `capacity`, 8 byte per il puntatore effettivo), per un totale di 24 byte.
 - `vslice`: *fat pointer* allo heap nella stessa zona di memoria di `vector`, contenente 8 byte per la dimensione dello slice e 8 byte per il puntatore, per un totale di 16 byte.
- **Heap:** Un `Vec` allocato di $8 \times \text{sizeof}(u64) = 8 \times 8 = 64$ byte.

Soluzione Domanda 2

Il problema di questo codice si pone in quanto il thread `writer` esegue solo 5 operazioni di scrittura, mentre i `reader` hanno un loop infinito. Di conseguenza, una volta consumati i 5 elementi dal writer, entrambi i reader entreranno in una `wait()` perenne non essendoci più nessuno che invocherà `notify_all()`, portando ad un classico caso di **deadlock** e impedendo al `main` di terminare, bloccato sulla `join()` dei thread reader.

Inoltre, con l'attuale configurazione le istruzioni alle righe 32 e 48 (`println!("{}", 1)` e `println!("{}", 2)`) verranno eseguite nei casi in cui i thread si sveglino a causa di *spurious wakeups* oppure in caso in cui un thread venga notificato ma, prima che riesca ad acquisire il lock e ad estrarre l'elemento, l'altro reader lo abbia già consumato lasciando il vettore vuoto.

Per risolvere questo problema vi sono due approcci principali:

1. **Utilizzare i canali MPMC/SPMC:** Invece di usare `Mutex` e `Condvar`, si potrebbe usare un canale di sincronizzazione (es. `std::sync::mpsc` o la crate `crossbeam`) che permetta al writer di inserire i dati e ai reader di accedere ad essi o mettersi in attesa. Terminando il suo ciclo, il writer deallocherebbe il `Sender`, causando la restituzione di un errore alle chiamate `recv()` dei `Receiver`. I reader potrebbero così uscire dal loop, terminare correttamente la propria esecuzione e sbloccare la `join()`.
2. **Aggiungere un flag di completamento (*poisoning/closing*):** Un'alternativa consiste nell'aggiungere al `Mutex`, oltre al vettore, un `bool` che indichi che il writer ha finito le scritture. In questo modo si modifica il tipo incapsulato in `Mutex<(Vec<i32>, bool)>`. I reader, ad ogni iterazione, subito dopo aver acquisito il lock (o al risveglio dalla `wait`), possono controllare tale booleano: se è `true` ed il vettore è vuoto, il thread esegue un `return` per terminare il ciclo ed il thread.

Esempio di correzione tramite flag booleano:

```

1 // Nel main inizializzo il mutex con una tupla
2 let data = Arc::new((Mutex::new((Vec::new(), false)), Condvar::new()));
3
4 // ... nel writer ...
5 // alla fine del loop
6 {
7     let mut data = lock.lock().unwrap();
8     data.1 = true;
9     cvar.notify_all(); // Risveglia i reader per fargli leggere il flag
10 }
11
12 // ... in entrambi i reader ...
13 loop {
14     let mut data = lock.lock().unwrap();
15
16     // Attesa con controllo della condizione (evita spurious wakeup)
17     while data.0.is_empty() && !data.1 {
18         data = cvar.wait(data).unwrap();
19     }
20
21     if let Some(value) = data.0.pop() {
22         println!("Reader: read {}", value);
23     } else if data.1 {
24         // Il writer ha finito e non ci sono piu' elementi da leggere
25         break;
26     }
27 }

```

Soluzione Domanda 3

Ecco un'implementazione thread-safe della struct `CountDownLatch` utilizzando un `Mutex` e una `Condvar`:

```

1 use std::sync::{Mutex, Condvar, WaitTimeoutResult};
2 use std::time::Duration;
3
4 pub struct CountDownLatch {
5     state: Mutex<usize>,
6     cvar: Condvar,
7 }
8
9 impl CountDownLatch {
10     pub fn new(n: usize) -> Self {
11         CountDownLatch {
12             state: Mutex::new(n),
13             cvar: Condvar::new(),
14         }
15     }
16
17     pub fn count_down(&self) {
18         let mut count = self.state.lock().unwrap();
19         if *count > 0 {
20             *count -= 1;
21             if *count == 0 {
22                 // Notifica tutti i thread in attesa quando il contatore arriva
23                 a zero
24                 self.cvar.notify_all();
25             }
26         }
27     }
28
29     pub fn wait(&self) {
30         let mut count = self.state.lock().unwrap();

```

```

30         // Ciclo while per gestire gli spurious wakeups
31         while *count > 0 {
32             count = self.cvar.wait(count).unwrap();
33         }
34     }
35
36     pub fn wait_timeout(&self, d: Duration) -> WaitTimeoutResult {
37         let count = self.state.lock().unwrap();
38         // wait_timeout_while attende finché la condizione (c > 0) è vera o il
39         // timeout scade
40         let (_guard, timeout_result) = self.cvar
41             .wait_timeout_while(count, d, |c| *c > 0)
42             .unwrap();
43         timeout_result
44     }
45 }

```

Soluzione Domanda 4

Il programma in questione inizializza un vettore di `i32`, inserisce un elemento, e poi definisce una closure denominata `process_data`. Siccome la closure utilizza la keyword `move`, essa **prende possesso** (*ownership*) delle variabili libere usate all'interno, ovvero del vettore `data`.

L'errore di compilazione Il programma in seguito prova ad inserire un nuovo elemento (`data.push(40)` alla riga 10), poi chiama `process_data()` (che inserisce 50 nel proprio vettore e conta gli elementi pari), ed infine prova ad inserire un ulteriore elemento (`data.push(30)` alla riga 14). L'errore di compilazione avviene perché, a partire dalla riga 10 (e poi 14), si cerca di utilizzare la variabile `data` di cui l'*ownership* è già stata trasferita (spostata) all'interno della closure `process_data` tramite la keyword `move`. Il compilatore Rust bloccherà l'operazione poiché l'uso di valori spostati (*use of moved value*) viola le regole di *ownership*.

La modifica Per risolvere il problema di compilazione bisogna passare alla closure un **clone** di `data`. Questo è fattibile in quanto `Vec` implementa il trait `Clone`. Così facendo, `process_data` prenderà l'*ownership* del clone, permettendo al codice principale di continuare ad operare sul `Vec` originale. Per ottenere esattamente in output `"Numero: 4"`, dobbiamo assicurarci che `process_data` operi su un vettore che contenga 3 elementi pari al suo interno, per un totale di 4 dopo il suo `push(50)`. Clonandolo subito prima della closure (quando il vettore è `[1, 2, 3, 4, 5, 60]`), i pari in esso contenuti sono 2, 4, e 60. Con l'aggiunta di 50, il conto dei numeri pari diventa esattamente 4.

La correzione del codice sarà:

```

1 fn main() {
2     let mut data = vec![1, 2, 3, 4, 5];
3     data.push(60);
4
5     // Cloniamo il dato prima della closure
6     let mut data_clone = data.clone();
7
8     // La closure ora cattura (e prende possesso) di data_clone
9     let mut process_data = move || {
10         data_clone.push(50);
11         let count = data_clone.iter().filter(|&x| x % 2 == 0).count();
12         println!("Numero: {:?}", count);
13     };
14
15     data.push(40);
16 }

```

```
17     process_data();  
18  
19     data.push(30);  
20 }
```